# LAB02: PRINCIPLE COMPONENT ANALYSIS (PCA)

*Ngày 29 tháng 4 năm 2026*

Người thực hiện: Mai Hoàng Hiệp
MSSV: 24122015

# Mục lục

**1 Câu 1: Cơ sở toán học của Phân tích thành phần chính (PCA) 4**
1.1 Xây dựng bài toán . . . . . 4
1.2 Các bước suy diễn toán học . . . . . 4
1.3 Kết luận . . . . . 5
1.4 Chứng minh . . . . . 7

**2 Câu 2 Lập trình cơ bản 11**
2.1 Các eigenvalue và eigenvector . . . . . 11
2.1.1 So sánh về Trị riêng . . . . . 12
2.1.2 So sánh về Vector riêng . . . . . 13
2.1.3 Kết luận . . . . . 13
2.2 Chiếu dữ liệu sang không gian mới . . . . . 14

**3 Câu 3 Trực quan hóa 15**
3.1 So sánh dữ liệu trước và sau PCA . . . . . 17
3.2 Nhận xét về phương sai . . . . . 18

**4 Câu 4 So sánh 19**
4.1 Đánh giá về Tốc độ chạy . . . . . 19
4.2 Đánh giá về Sai số giữa hai cách . . . . . 20

**5 Câu 5 Ứng dụng PCA vào bài toán nén và tái tạo ảnh 21**
5.1 Ảnh hưởng của số lượng thành phần chính ($k$) đối với chất lượng ảnh: . . . . . 21
5.2 Sự đánh đổi (Trade-off) giữa Độ nén và Chất lượng ảnh: . . . . . 21

**6 Câu hỏi Bonus 22**
6.1 So sánh PCA và t-SNE, Linear Discriminant Analysis . . . . . 22
6.1.1 Phân tích PCA và t-SNE . . . . . 22
6.1.2 Phân tích PCA và LDA . . . . . 23
6.2 Áp dụng PCA cho bài toán Giảm nhiễu tín hiệu (Signal/Image Denoising) . . . . 23
6.3 Áp dụng PCA cho bài toán nhận diện khuôn mặt . . . . . 24
6.4 Giải quyết vấn đề về bài toán tràn bộ nhớ bằng thủ thuật Turk Pentland . . . . 24
6.5 Tích hợp lọc nhiễu tín hiệu . . . . . 25
6.6 Đánh giá kết quả Kiểm thử và Tinh chỉnh tham số k-NN . . . . . 25

# Danh sách hình ảnh

1 Kết quả in Trị riêng và Vector riêng từ code tự cài đặt bằng NumPy . . . . . 11
2 Kết quả in Trị riêng và Vector riêng từ code của thư viện scikit-learn . . . . . 12
3 Kết quả khi chiếu dữ liệu sang không gian mới . . . . . 14
4 Hình ảnh dữ liệu gốc sau khi chuẩn hóa . . . . . 15
5 Hình ảnh về Dữ liệu gốc và các thành phần chính . . . . . 16
6 Hình ảnh về Dữ liệu sau khi chiếu xuống 1D . . . . . 16
7 Minh họa không gian dữ liệu gốc và kết quả thu được sau khi thực hiện phép chiếu lên thành phần chính thứ nhất . . . . . 17
8 So sánh tốc độ chạy và sai số giữa PCA khi tự code và dùng thư viện . . . . . 19
9 So sánh ảnh gốc và các ảnh được tái tạo . . . . . 21
10 Phân rã khuôn mặt thành các đặc trưng toán học PCA. . . . . 24
11 Kết quả kiểm thử sau quá trình huấn luyện . . . . . 26

# 1 Câu 1: Cơ sở toán học của Phân tích thành phần chính (PCA)

## 1.1 Xây dựng bài toán

**Bài toán giảm chiều dữ liệu:** Trong thực tế, dữ liệu thường nằm ở không gian nhiều chiều $D$, nơi chứa nhiều đặc trưng dư thừa hoặc nhiễu. Mục tiêu của việc giảm chiều dữ liệu là tìm một phép biến đổi để đưa tập dữ liệu $X$ (gồm $N$ điểm dữ liệu $x_i \in \mathbb{R}^D$) xuống một không gian mới có số chiều $K$ nhỏ hơn ($K < D$), sao cho lượng thông tin quan trọng nhất của tập dữ liệu ban đầu vẫn được giữ lại.

**Mục tiêu tối đa hóa phương sai:** Đối với thống kê, lượng thông tin của dữ liệu được biểu thị thông qua mức độ phân tán, còn gọi là **phương sai**. Vì vậy, mục tiêu của bài toán PCA là tìm ra một không gian con (được định nghĩa bởi các vector chỉ phương) sao cho khi ta chiếu các điểm dữ liệu gốc lên không gian con này, phương sai của các điểm dữ liệu sau khi chiếu đạt giá trị lớn nhất.

## 1.2 Các bước suy diễn toán học

**Chuẩn hóa dữ liệu (mean-centering):** Chuẩn hóa dữ liệu (mean-centering): Để các tính toán phương sai không bị ảnh hưởng bởi vị trí của dữ liệu trong không gian, ta sẽ thực hiện chuẩn hóa dũ liệu, ở đây, ta cần dời tâm của toàn bộ tập dữ liệu về gốc tọa độ.

- *Kỳ vọng của tập dữ liệu:* $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$
- *Thực hiện chuẩn hóa:* $x_i \leftarrow x_i - \mu$. Sau bước này, trung bình của tập dữ liệu mới bằng 0.

**Xây dựng ma trận hiệp phương sai:** Dữ liệu sau khi chuẩn hóa sẽ được dùng để tính mức độ tương quan giữa các chiều đặc trưng với nhau thông qua ma trận hiệp phương sai $S$.

$$S = \frac{1}{N}\sum_{i=1}^{N} x_i x_i^T \tag{1}$$

*(Với $S$ là ma trận vuông kích thước $D \times D$, đối xứng và bán xác định dương).*

**Bài toán tối ưu:** Gọi $u$ là một vector chỉ phương trong không gian gốc sao cho độ dài của nó bằng 1 ($\|u\|_2 = 1 \iff u^T u = 1$). Tọa độ của một điểm $x_i$ khi chiếu lên hướng $u$ là vô hướng $u^T x_i$. Phương sai của dữ liệu trên hướng $u$ là:

$$Var(u) = \frac{1}{N}\sum_{i=1}^{N}(u^T x_i)^2 = \frac{1}{N}\sum_{i=1}^{N}(u^T x_i)(x_i^T u) = u^T\left(\frac{1}{N}\sum_{i=1}^{N} x_i x_i^T\right)u = u^T S u \tag{2}$$

Mục tiêu là tìm vector $u$ để tối đa hóa phương sai này, dẫn đến bài toán tối ưu có ràng buộc:

$$\max_u (u^T S u) \text{ với điều kiện } u^T u = 1$$

**Sử dụng nhân tử Lagrange để đưa về bài toán trị riêng:** Ta sử dụng phương pháp nhân tử Lagrange để giải quyết bài toán tối ưu có ràng buộc trên bằng cách thiết lập hàm Lagrange

với nhân tử $\lambda$:

$$\mathcal{L}(u, \lambda) = u^T S u - \lambda(u^T u - 1) \tag{3}$$

Để tìm điểm cực trị, ta lấy đạo hàm riêng của $\mathcal{L}$ theo vector $u$ và cho bằng 0:

$$\frac{\partial \mathcal{L}}{\partial u} = 2Su - 2\lambda u = 0 \tag{4}$$

Chia cả hai vế của (4) cho 2, ta thu được phương trình:

$$Su = \lambda u$$

Phương trình này chứng minh bài toán tối ưu hóa ban đầu đã được đưa về việc tìm trị riêng ($\lambda$) và vector riêng ($u$) của ma trận hiệp phương sai $S$.

## 1.3 Kết luận

**Giải thích vì sao PCA tương đương với việc tìm eigenvectors của ma trận hiệp phương sai**

*Nhắc lại mục tiêu*: Cốt lõi của PCA là tìm một hướng (đại diện bởi vector chỉ phương $u$ với điều kiện $u^T u = 1$) sao cho khi chiếu dữ liệu lên hướng đó, phương sai đạt giá trị lớn nhất. Biểu thức cần tối đa hóa là: $\max_u u^T S u$ (trong đó $S$ là ma trận hiệp phương sai). *Liên kết với Đại số tuyến tính*: Khi giải bài toán tối ưu trên bằng nhân tử Lagrange (đã chứng minh ở phần b), ta thu được điều kiện tại điểm cực trị là:

$$Su = \lambda u$$

Theo định nghĩa trong đại số tuyến tính, phương trình này chính là bài toán tìm trị riêng và vector riêng. Cụ thể, đối với ma trận vuông $S$, $u$ được gọi là vector riêng (eigenvector) và đại lượng vô hướng $\lambda$ được gọi là trị riêng (eigenvalue) tương ứng. *Chứng minh sự tương đương:* ta thay trực tiếp phương trình $Su = \lambda u$ ngược lại vào công thức tính phương sai ban đầu:

$$Var(u) = u^T S u = u^T(\lambda u) = \lambda(u^T u)$$

Vì ta đã đặt điều kiện vector chỉ phương có độ dài bằng 1 ($u^T u = 1$), phương trình trên được rút gọn thành:

$$Var(u) = \lambda \cdot 1 = \lambda$$

Qua đây, ta chứng minh được rằng phương sai của dữ liệu chiếu trên hướng $u$ có giá trị bằng đúng trị riêng $\lambda$. Do đó, để tìm được các hướng tối đa hóa phương sai (mục tiêu của PCA), ta bắt buộc phải đi tìm các vector riêng (eigenvectors) của ma trận hiệp phương sai $S$. Bài toán tối ưu PCA và bài toán tìm eigenvectors hoàn toàn là một.

**Ý nghĩa của eigenvalues (phương sai giữ lại)** Dựa vào kết quả đã chứng minh ở phần trước là $Var(u) = \lambda$, ta có thể rút ra những ý nghĩa vô cùng quan trọng của trị riêng (eigenvalues) trong phân tích thành phần chính như sau:

- Đại diện cho lượng thông tin (Phương sai): Trong lý thuyết thông tin và thống kê, phương sai đo lường mức độ phân tán của dữ liệu. Dữ liệu phân tán càng rộng trên một trục tọa độ thì trục đó càng chứa nhiều thông tin. Vì $Var(u) = \lambda$, trị riêng $\lambda$ chính là độ lớn của phương sai của tập dữ liệu khi được chiếu lên vector riêng (thành phần chính) $u$ tương ứng.
- Tỷ lệ phương sai giữ lại (Explained Variance Ratio): Giả sử ma trận hiệp phương sai của dữ liệu $D$ chiều có $D$ trị riêng là $\lambda_1, \lambda_2, \ldots, \lambda_D$. Tổng phương sai của toàn bộ dữ liệu gốc bằng tổng của tất cả các trị riêng: $V_{total} = \sum_{j=1}^{D} \lambda_j$.Từ đó, tỷ lệ phương sai (lượng thông tin) được giữ lại bởi thành phần chính thứ $i$ được tính bằng công thức:

$$\text{Tỷ lệ} = \frac{\lambda_i}{\sum_{j=1}^{D} \lambda_j}$$

- Cơ sở để loại bỏ nhiễu và giảm chiều: Nếu một $\lambda_i$ có giá trị rất lớn, vector riêng tương ứng của nó là một hướng quan trọng, cấu trúc nên hình dạng chính của dữ liệu. Ngược lại, nếu $\lambda_i$ có giá trị rất nhỏ (tiến sát về 0), sự biến thiên của dữ liệu dọc theo hướng đó gần như không đáng kể. Hướng này thường chỉ chứa nhiễu (noise) hoặc các đặc trưng thừa, và có thể bị loại bỏ mà không làm mất đi các đặc tính quan trọng của tập dữ liệu gốc.

**Sắp xếp thành phần chính:** Sau khi giải phương trình đặc trưng $Su = \lambda u$ của ma trận hiệp phương sai, ta thu được $D$ cặp (trị riêng, vector riêng) tương ứng với $D$ chiều của không gian gốc. Việc sắp xếp các thành phần chính là bước quyết định để thực hiện giảm chiều dữ liệu.

- Quy tắc sắp xếp: Ta tiến hành sắp xếp các trị riêng thu được theo thứ tự giảm dần:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$$

  Đồng thời, các vector riêng tương ứng cũng được hoán vị để khớp đúng với thứ tự của trị riêng: $u_1, u_2, \ldots, u_D$.

- Mục đích của việc sắp xếp: Việc sắp xếp này thiết lập một thứ tự ưu tiên về lượng thông tin đóng góp của từng chiều trong không gian mới:
  - Thành phần chính thứ nhất (PC1 - $u_1$): Ứng với $\lambda_1$ lớn nhất, đây là trục tọa độ mà khi chiếu dữ liệu lên đó, độ phân tán (phương sai) là cực đại. Trục này giữ lại nhiều thông tin cấu trúc của dữ liệu gốc nhất.
  - Thành phần chính thứ hai (PC2 - $u_2$): Ứng với $\lambda_2$ lớn thứ hai, đây là hướng vuông góc (trực giao) với PC1 và giữ lại lượng phương sai nhiều thứ hai.
  - Tương tự cho các thành phần tiếp theo, mức độ quan trọng giảm dần.
- Ứng dụng vào giảm chiều dữ liệu ($K < D$): Nhờ có sự sắp xếp này, khi bài toán yêu cầu giảm xuống còn $K$ chiều, ta không chọn ngẫu nhiên mà sẽ cắt lấy đúng $K$ vector riêng đầu tiên $(u_1, u_2, \ldots, u_K)$ để tạo thành ma trận cơ sở cho không gian mới. Phép chiếu lên không gian con sinh bởi $K$ vector này được đảm bảo toán học là cách nén dữ liệu tối ưu

tuyến tính tốt nhất, vì nó tối đa hóa tỷ lệ phương sai tích lũy được giữ lại:

$$\text{Tỷ lệ phương sai giữ lại} = \frac{\sum_{i=1}^{K} \lambda_i}{\sum_{j=1}^{D} \lambda_j}$$

## 1.4 Chứng minh

**Chứng minh vector riêng ứng với trị riêng lớn nhất là hướng tối đa hóa phương sai**
Mục tiêu: Cần chứng minh rằng bài toán tìm $\max_u(u^T S u)$ với điều kiện $u^T u = 1$ sẽ đạt giá trị lớn nhất tại $u = q_1$, trong đó $q_1$ là vector riêng ứng với trị riêng lớn nhất $\lambda_1$ của ma trận hiệp phương sai $S$. Chứng minh chi tiết

- Bước 1: Phân tích ma trận hiệp phương sai $S$ Ma trận hiệp phương sai $S$ (kích thước $D \times D$) là một ma trận đối xứng thực. Theo định lý phổ (Spectral Theorem) trong đại số tuyến tính, mọi ma trận đối xứng thực đều có thể chéo hóa trực giao. Cụ thể, ta có thể viết:

  $$S = Q\Lambda Q^T$$

  Với:

  - $Q$ là ma trận trực giao ($Q^T Q = QQ^T = I$), trong đó mỗi cột của $Q$ là một vector riêng $q_i$ của $S$.
  - $\Lambda$ là ma trận đường chéo chứa các trị riêng $\lambda_i$ của $S$. Ta giả sử các trị riêng đã được sắp xếp giảm dần: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D \geq 0$

- Bước 2: Biến đổi hàm mục tiêu Ta thay phân tích phổ của $S$ vào biểu thức phương sai $Var(u) = u^T S u$:

  $$u^T S u = u^T (Q\Lambda Q^T) u = (u^T Q)\Lambda(Q^T u)$$

  Đặt $v = Q^T u$. Vì $Q$ là ma trận trực giao, độ dài của vector $v$ bằng đúng độ dài của vector $u$:

  $$\|v\|^2 = v^T v = (Q^T u)^T (Q^T u) = u^T Q Q^T u = u^T I u = u^T u = 1$$

  Lúc này, hàm mục tiêu được viết lại theo $v$:

  $$u^T S u = v^T \Lambda v$$

  Do $\Lambda$ là ma trận đường chéo chứa các $\lambda_i$, biểu thức trên trở thành một tổng đại số:

  $$v^T \Lambda v = \sum_{i=1}^{D} \lambda_i v_i^2$$

  (với $v_i$ là các thành phần của vector $v$).

- Bước 3: Đánh giá bất đẳng thức Vì $\lambda_1$ là trị riêng lớn nhất, ta có $\lambda_i \leq \lambda_1$ với mọi

$i = 1, 2, \ldots, D$. Do $v_i^2 \geq 0$, ta thiết lập được bất đẳng thức sau:

$$\sum_{i=1}^{D} \lambda_i v_i^2 \leq \sum_{i=1}^{D} \lambda_1 v_i^2$$

Rút $\lambda_1$ ra làm nhân tử chung ở vế phải:

$$\sum_{i=1}^{D} \lambda_1 v_i^2 = \lambda_1 \left( \sum_{i=1}^{D} v_i^2 \right)$$

Mà theo định nghĩa ở bước 2, ta đã có $\sum_{i=1}^{D} v_i^2 = \|v\|^2 = 1$. Do đó:

$$u^T S u = \sum_{i=1}^{D} \lambda_i v_i^2 \leq \lambda_1 \cdot 1 = \lambda_1$$

Vậy, giá trị lớn nhất của phương sai chính là $\lambda_1$.

- Bước 4: Tìm điều kiện để dấu "=" xảy ra (Xác định hướng tối ưu) Dấu "=" trong bất đẳng thức trên xảy ra khi và chỉ khi:

  $$\sum_{i=1}^{D} (\lambda_1 - \lambda_i) v_i^2 = 0$$

  Vì $\lambda_1 - \lambda_i \geq 0$ và $v_i^2 \geq 0$, điều này ép buộc $v_i = 0$ với mọi $i > 1$.

  Kết hợp với điều kiện $\|v\|^2 = 1$, ta suy ra $v_1^2 = 1$ (hay $v_1 = 1$, xét theo một chiều vector). Vậy vector $v$ có dạng: $v = [1, 0, \ldots, 0]^T$.

  Trở lại với phép đặt $v = Q^T u$, ta tìm được $u$:

  $$u = Qv = [q_1, q_2, \ldots, q_D] \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = q_1$$

Kết luận: Phương sai đạt giá trị cực đại bằng đúng $\lambda_1$ khi vector chỉ phương $u$ chính là $q_1$. Điều này chứng tỏ một cách chặt chẽ toán học rằng vector riêng ứng với trị riêng lớn nhất chính là hướng giúp tối đa hóa phương sai của dữ liệu.

**Chứng minh tính trực giao của các principal components** Mục tiêu: Cho ma trận hiệp phương sai $S$. Giả sử $u_i$ và $u_j$ là hai vector riêng (principal components) ứng với hai trị riêng phân biệt $\lambda_i$ và $\lambda_j$ (nghĩa là $\lambda_i \neq \lambda_j$). Ta cần chứng minh $u_i$ và $u_j$ trực giao với nhau, tức là tích vô hướng của chúng bằng 0: $u_j^T u_i = 0$. Chứng minh:

- Bước 1: Thiết lập phương trình trị riêng Theo định nghĩa của trị riêng và vector riêng đối

với ma trận $S$, ta có hệ phương trình:

$$Su_i = \lambda_i u_i \tag{5}$$

$$Su_j = \lambda_j u_j \tag{6}$$

- Bước 2: Sử dụng tính chất đối xứng của ma trận hiệp phương sai Ma trận hiệp phương sai $S$ được tính bằng công thức $S = \frac{1}{N}\sum xx^T$, do đó $S$ luôn là một ma trận đối xứng thực. Tính chất cốt lõi của ma trận đối xứng là nó bằng chính chuyển vị của nó:

$$S = S^T$$

- Bước 3: Biến đổi các phương trình Nhân cả hai vế của phương trình (5) từ bên trái với vector hàng $u_j^T$, ta được một đại lượng vô hướng:

$$u_j^T S u_i = \lambda_i u_j^T u_i \tag{7}$$

Nhân cả hai vế của phương trình (2) từ bên trái với vector hàng $u_i^T$:

$$u_i^T S u_j = \lambda_j u_i^T u_j$$

Vì cả hai vế đều là một số thực (vô hướng), ta có thể lấy chuyển vị của toàn bộ phương trình mà giá trị không thay đổi (chuyển vị của một số chính là số đó: $a^T = a$):

$$(u_i^T S u_j)^T = (\lambda_j u_i^T u_j)^T$$

Áp dụng quy tắc chuyển vị của tích các ma trận $(ABC)^T = C^T B^T A^T$, ta khai triển vế trái và vế phải:

$$u_j^T S^T (u_i^T)^T = \lambda_j u_j^T (u_i^T)^T$$
$$\implies u_j^T S^T u_i = \lambda_j u_j^T u_i$$

Do $S = S^T$ (đã nêu ở Bước 2), ta thế $S$ vào phương trình trên:

$$u_j^T S u_i = \lambda_j u_j^T u_i \tag{8}$$

- Bước 4: Kết luận từ hai phương trình Quan sát phương trình (7) và (8), ta thấy vế trái của chúng hoàn toàn giống nhau ($u_j^T S u_i$). Do đó, ta có thể cho hai vế phải bằng nhau:

$$\lambda_i u_j^T u_i = \lambda_j u_j^T u_i$$

Chuyển vế và đặt nhân tử chung, ta được:

$$(\lambda_i - \lambda_j) u_j^T u_i = 0$$

Vì giả thiết ban đầu của chúng ta là hai trị riêng phân biệt ($\lambda_i \neq \lambda_j$), nên đại lượng

$(\lambda_i - \lambda_j)$ chắc chắn khác 0. Để tích trên bằng 0, bắt buộc phần còn lại phải bằng 0:

$$u_j^T u_i = 0$$

*Lưu ý: Trong trường hợp ma trận $S$ có các trị riêng bị lặp (nghĩa là $\lambda_i = \lambda_j$ dù $u_i$ và $u_j$ là 2 vector khác nhau), theo Định lý Phổ (Spectral Theorem) của đại số tuyến tính, đối với không gian con sinh bởi các vector riêng ứng với cùng một trị riêng (eigenspace), ta luôn có thể sử dụng quá trình trực giao hóa Gram-Schmidt để chọn ra một hệ cơ sở trực giao. Do đó, các principal components luôn được đảm bảo tính trực giao với nhau.*

Kết luận: Ta đã chứng minh thành công rằng các thành phần chính (principal components) vuông góc với nhau, đảm bảo không gian biến đổi của PCA bảo toàn tính độc lập giữa các trục tọa độ.

# 2 Câu 2 Lập trình cơ bản

## 2.1 Các eigenvalue và eigenvector

Dưới đây là hình ảnh eigenvector và eigenvalue của ma trận 5x10, và bảng so sánh với thuật toán của tôi (sử dụng thư viện numpy), và hình ảnh eigenvector và eigenvalue của ma trận đó khi dùng thuật toán PCA của thư viện scikit-learn

```
Bước 3: Eigenvalues và Eigenvectors sau khi sắp xếp giảm dần
1. Trị riêng (Eigenvalues) đã sắp xếp giảm dần:
[ 5.85336433e-01  1.18381740e-01  9.35564979e-02  6.05969718e-02
  1.60182412e-17  6.68293365e-18 -6.68472594e-19 -2.98191602e-18
 -2.76105076e-17 -1.35930547e-16]

2. Vector riêng (Eigenvectors) tương ứng:
[[-2.33552091e-01 -6.27203269e-02 -5.91821035e-01  3.81593066e-01
   5.15095797e-02 -4.99797154e-04  7.88327817e-02  7.44505360e-02
  -2.90159713e-01  5.89109957e-01]
 [ 5.05470231e-01 -2.69045634e-01  1.46760053e-01 -2.20183187e-01
  -7.56578107e-02  9.76744237e-02 -1.20536634e-01  1.09198073e-01
   3.67806507e-01  6.51993726e-01]
 [ 4.48106297e-01  7.59903402e-02 -4.43729223e-01 -1.79116058e-01
  -9.11211970e-02 -3.49875627e-01  2.24492865e-02 -6.49501626e-01
  -6.11571508e-02 -8.73811913e-02]
 [-2.65822600e-01 -6.41326438e-01  3.97726552e-01  1.03858208e-01
  -2.51320097e-01 -1.52242232e-01  3.57808636e-02 -4.30129742e-01
  -2.56229229e-01  1.03831766e-01]
 [-3.76428841e-01 -1.96120397e-01 -3.88537646e-01 -4.24186903e-01
  -5.11640056e-01  1.17838636e-01  2.38318968e-01  9.52653000e-02
   3.69994671e-01 -1.02533561e-01]
 [-4.11378463e-01  1.89444270e-01  8.09506621e-02  1.27145157e-01
   3.68588348e-01 -1.06917482e-01 -1.83998587e-02 -4.32276587e-01
   6.32922659e-01  1.92557304e-01]
 [-6.19408729e-02  8.63961034e-02  1.70389584e-01 -3.19144730e-01
   2.00485019e-01 -6.18497541e-01  5.71966572e-01  2.50068517e-01
  -1.12639870e-01  1.80864201e-01]
 [ 2.82582725e-01 -1.17546325e-02  5.16290510e-02  6.70712773e-01
  -3.27628804e-01 -1.20345651e-01  4.50675272e-01  8.82801321e-02
   3.35228853e-01 -1.49612749e-01]
 [-8.56155875e-02 -1.35284878e-01 -1.11560411e-01  1.22325149e-01
  -1.34163428e-01 -6.47425479e-01 -6.10915675e-01  3.11861914e-01
   1.69735992e-01 -1.02533448e-01]
 [ 1.28614733e-01 -6.37455628e-01 -2.67336697e-01  1.62530803e-02
   5.97486549e-01  5.80472164e-02  1.29867875e-01  1.10203987e-01
   1.40600565e-01 -3.10220474e-01]]

 ------------------------------------------------
```

Hình 1: Kết quả in Trị riêng và Vector riêng từ code tự cài đặt bằng NumPy

```
--------------------------------------------------

 SO SÁNH VỚI SCIKIT-LEARN PCA CÓ SẴN
Scikit-learn - Trị riêng (explained_variance_):
[5.85336433e-01 1.18381740e-01 9.35564979e-02 6.05969718e-02
 9.18247286e-33]

Scikit-learn - Vector riêng (components_.T):
[[-0.23355209  0.06272033  0.59182103  0.38159307 -0.09474751]
 [ 0.50547023  0.26904563 -0.14676005 -0.22018319  0.19530281]
 [ 0.4481063  -0.07599034  0.44372922 -0.17911606  0.19862071]
 [-0.2658226   0.64132644 -0.39772655  0.10385821 -0.08792029]
 [-0.37642884  0.1961204   0.38853765 -0.4241869  -0.22277209]
 [-0.41137846 -0.18944427 -0.08095066  0.12714516  0.57912041]
 [-0.06194087 -0.0863961  -0.17038958 -0.31914473  0.17066145]
 [ 0.28258272  0.01175463 -0.05162905  0.67071277 -0.11145404]
 [-0.08561559  0.13528488  0.11156041  0.12232515  0.67297069]
 [ 0.12861473  0.63745563  0.2673367   0.01625308  0.16203341]]

--------------------------------------------------
```

Hình 2: Kết quả in Trị riêng và Vector riêng từ code của thư viện scikit-learn

Dựa trên kết quả xuất ra từ quá trình chạy thuật toán PCA tự cài đặt bằng NumPy và hàm PCA có sẵn của thư viện scikit-learn, ta có thể rút ra những đánh giá và phân tích chuyên sâu về mặt đại số tuyến tính như sau:

### 2.1.1 So sánh về Trị riêng

**Sự tương đồng (Tính chính xác):**

Bốn trị riêng đầu tiên của cả hai phương pháp cho ra kết quả trùng khớp hoàn toàn đến từng chữ số thập phân (lần lượt là $\approx 0.585, 0.118, 0.093, 0.060$). Điều này chứng minh quá trình xây dựng ma trận hiệp phương sai và giải hệ phương trình đặc trưng trong code tự cài đặt là hoàn toàn chính xác.

**Sự khác biệt 1: Số lượng trị riêng được trả về (10 so với 5):**

- NumPy tự cài đặt: Trả về đầy đủ 10 trị riêng. Do ma trận hiệp phương sai có kích thước $D \times D$ ($10 \times 10$), hàm np.linalg.eigh giải quyết trọn vẹn không gian này nên sinh ra 10 nghiệm.
- scikit-learn: Chỉ trả về 5 trị riêng. Nguyên nhân là do hàm PCA của scikit-learn giới hạn số lượng thành phần chính tối đa bằng $\min(N, D) = \min(5, 10) = 5$.
- Giải thích toán học: Vì tập dữ liệu chỉ có $N = 5$ mẫu, sau khi chuẩn hóa (mean-centering) sẽ mất 1 bậc tự do. Do đó, các điểm dữ liệu này chỉ có thể nằm gọn trong một không gian con tối đa 4 chiều (Hạng ma trận hiệp phương sai $Rank(S) = \min(N-1, D) = 4$). Về mặt lý thuyết, chỉ có 4 trị riêng mang giá trị dương, tất cả các chiều còn lại không chứa thông tin (phương sai bằng 0). Việc scikit-learn cắt bỏ bớt giúp tối ưu hóa bộ nhớ và thời gian tính toán.

**Sự khác biệt 2: Sai số làm tròn của số 0 (e-17 so với e-33):**

Từ trị riêng thứ 5 trở đi, theo lý thuyết toán học phải bằng chính xác 0. Tuy nhiên:

- NumPy cho ra kết quả ở mức $10^{-17}$ (ví dụ: $1.60 \times 10^{-17}$).
- scikit-learn cho ra kết quả ở mức $10^{-33}$ (ví dụ: $9.18 \times 10^{-33}$).
- Giải thích: Thuật toán tự cài đặt phải tính ma trận hiệp phương sai thông qua phép nhân ma trận $X^T X$, điều này làm khuyếch đại sai số làm tròn số thập phân (floating-point error) của máy tính. Trong khi đó, scikit-learn sử dụng thuật toán SVD (Singular Value Decomposition) trực tiếp trên ma trận $X$ mà không cần tính $X^T X$, do đó độ ổn định số học cao hơn rất nhiều, giúp "số 0" tiến sát về độ chính xác tuyệt đối hơn.

### 2.1.2 So sánh về Vector riêng

**Sự tương đồng về giá trị tuyệt đối:**

Khi đối chiếu từng cột của ma trận vector riêng (ở NumPy) và từng cột tương ứng (ở dạng chuyển vị .T của scikit-learn), ta thấy các giá trị tuyệt đối hoàn toàn trùng khớp nhau. Điều này khẳng định hai phương pháp đã tìm ra cùng một không gian cơ sở trực giao để thực hiện phép chiếu.

**Sự khác biệt: Hiện tượng ngược dấu (Sign Ambiguity):**

Quan sát kỹ, ví dụ ở vector riêng thứ 2 và thứ 3, toàn bộ các phần tử của NumPy và scikit-learn bị ngược dấu nhau hoàn toàn (nhân với $-1$) Giải thích toán học: Đây là tính chất Không duy nhất của Vector riêng (Sign Ambiguity). Trong đại số tuyến tính, phương trình trị riêng được định nghĩa là $Sv = \lambda v$. Nếu ta nhân cả hai vế với $-1$, ta được $S(-v) = \lambda(-v)$. Điều này chứng tỏ nếu $v$ là một vector riêng hợp lệ, thì vector đối của nó là $-v$ (cùng phương, ngược chiều) cũng là một vector riêng hoàn toàn hợp lệ ứng với cùng một trị riêng $\lambda$. Lý do lập trình: Hàm np.linalg.eigh của thư viện NumPy giải ma trận dựa trên các giá trị khởi tạo ngẫu nhiên ở tầng thấp (C/Fortran), nên chiều của vector có thể rơi vào cực âm hoặc dương ngẫu nhiên. Ngược lại, scikit-learn có tích hợp hàm svd_flip để chuẩn hóa và ép buộc vector riêng luôn tuân theo một quy tắc dấu cố định (đảm bảo kết quả deterministic ở mọi lần chạy).

### 2.1.3 Kết luận

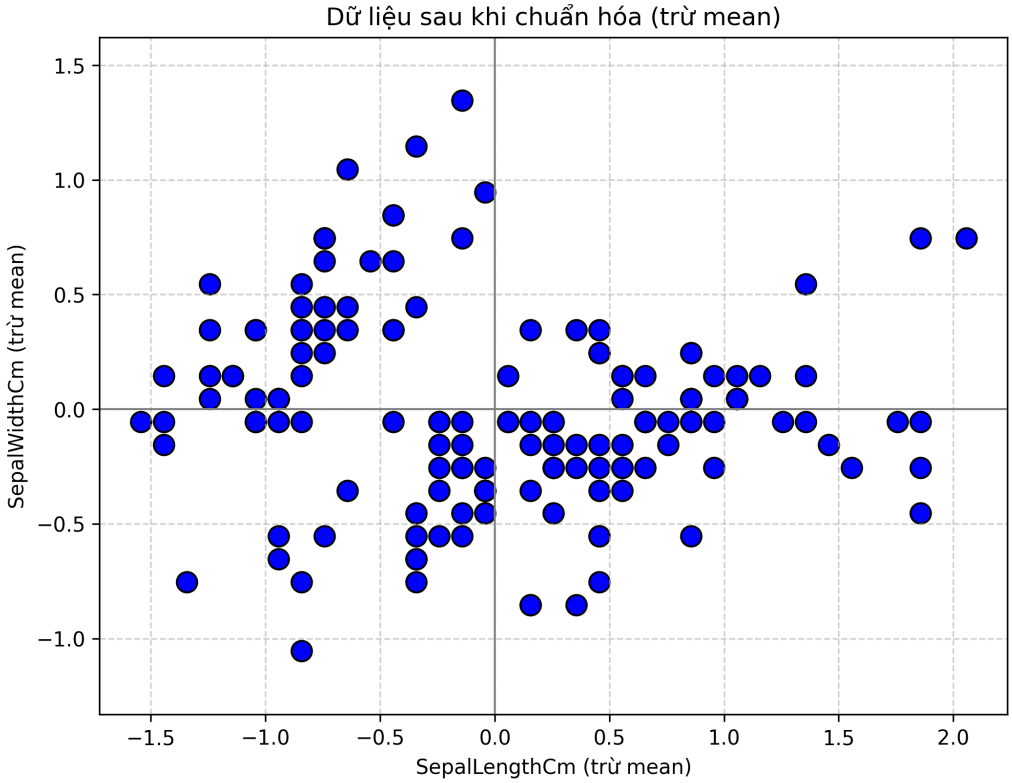
Thuật toán PCA tự cài đặt bằng các phép toán ma trận cơ sở của NumPy hoạt động chính xác và tương đương 100% về mặt hình học so với thuật toán được tối ưu hóa trong thư viện công nghiệp scikit-learn. Những khác biệt nhỏ xuất hiện hoàn toàn là do đặc tính của Đại số tuyến tính (hạng ma trận, tính đa trị của vector riêng) và cách tối ưu hóa giải tích số (SVD) của các công cụ.

## 2.2 Chiếu dữ liệu sang không gian mới

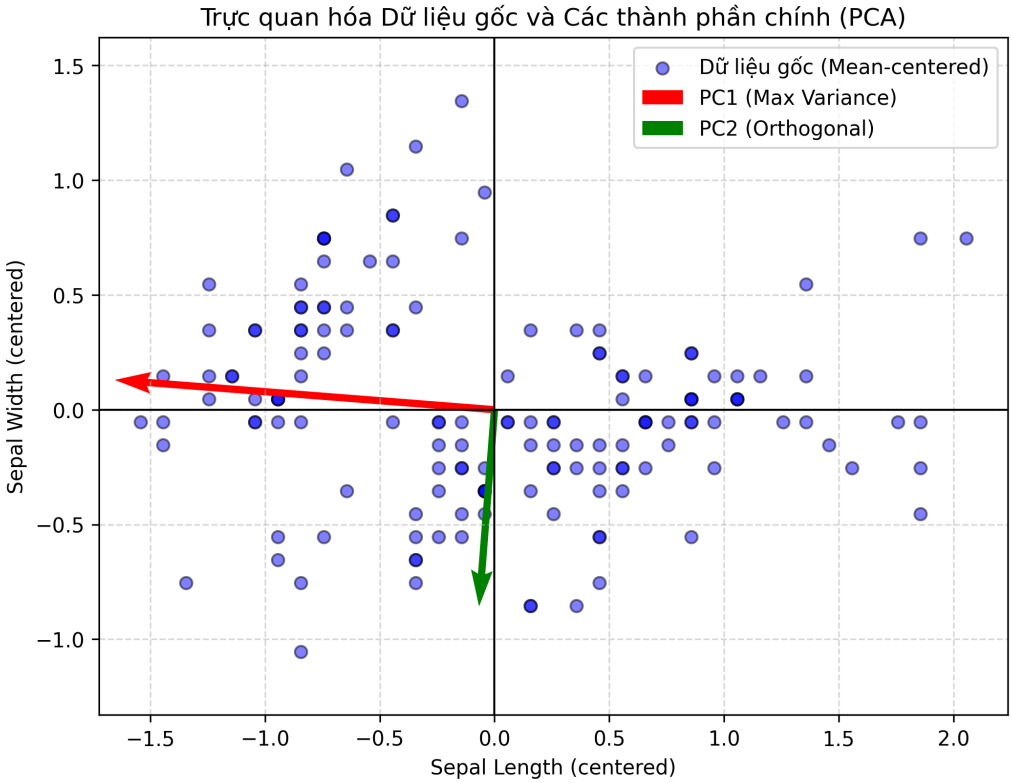
```
Bước 5: Chiếu dữ liệu xuống không gian mới với các giá trị k khác nhau

 - GIẢM XUỐNG CÒN k = 1 CHIỀU
Kích thước ma trận sau khi chiếu: (5, 1)

 - GIẢM XUỐNG CÒN k = 2 CHIỀU
Kích thước ma trận sau khi chiếu: (5, 2)
[[ 0.75490422 -0.36170637]
 [ 0.82313285  0.22473659]
 [-0.41027001  0.45686852]
 [-0.932994   -0.28701722]
 [-0.23477307 -0.03288153]]

 - GIẢM XUỐNG CÒN k = 3 CHIỀU
Kích thước ma trận sau khi chiếu: (5, 3)

 - GIẢM XUỐNG CÒN k = 4 CHIỀU
Kích thước ma trận sau khi chiếu: (5, 4)

 - GIẢM XUỐNG CÒN k = 5 CHIỀU
Kích thước ma trận sau khi chiếu: (5, 5)
--------------------------------------------------
=== VÍ DỤ CHIẾU 1 VECTOR ĐƠN LẺ (Chọn k = 2) ===
Vector gốc ban đầu (10 chiều):
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864 0.15599452
 0.05808361 0.86617615 0.60111501 0.70807258]

Vector sau khi chiếu xuống không gian (k=2 chiều):
[ 0.75490422 -0.36170637]
```

Hình 3: Kết quả khi chiếu dữ liệu sang không gian mới

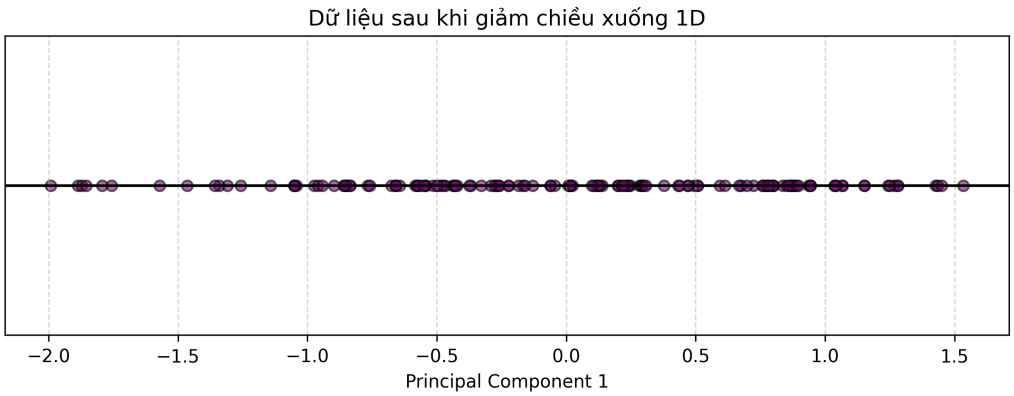
# 3 Câu 3 Trực quan hóa



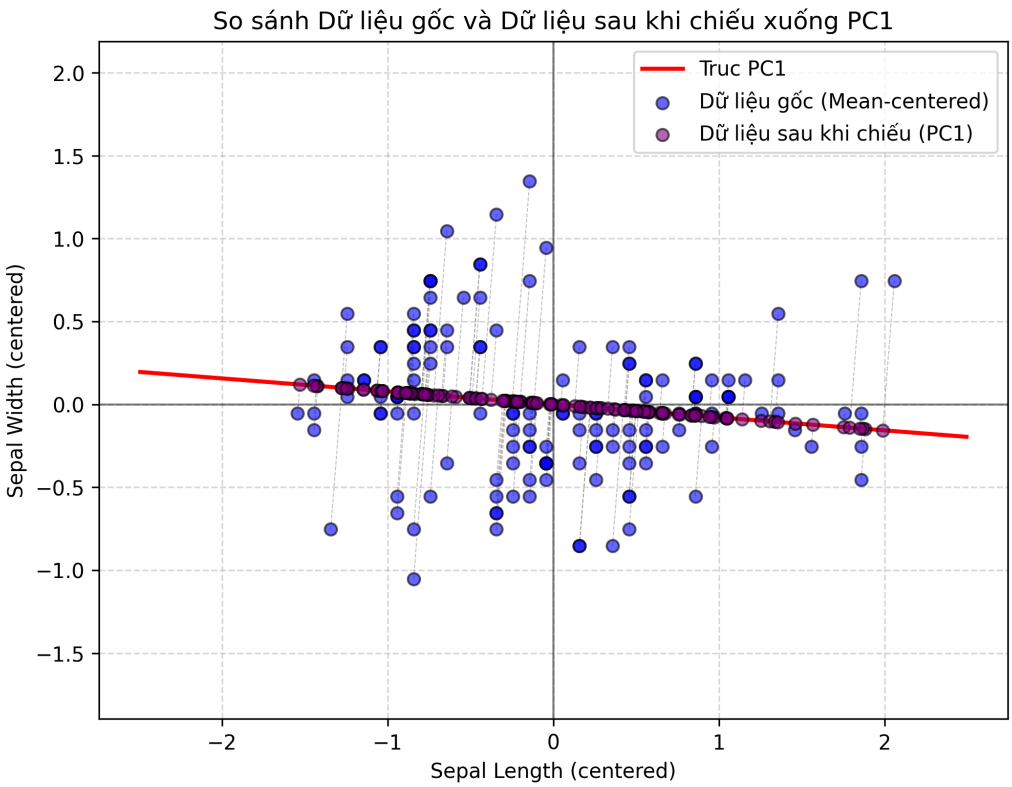

Hình 4: Hình ảnh dữ liệu gốc sau khi chuẩn hóa




Hình 5: Hình ảnh về Dữ liệu gốc và các thành phần chính




Hình 6: Hình ảnh về Dữ liệu sau khi chiếu xuống 1D




Hình 7: Minh họa không gian dữ liệu gốc và kết quả thu được sau khi thực hiện phép chiếu lên thành phần chính thứ nhất

## 3.1 So sánh dữ liệu trước và sau PCA

**Không gian dữ liệu gốc (Trạng thái trước PCA):**

Ở đồ thị đầu tiên, dữ liệu ban đầu tồn tại dưới dạng một đám mây điểm phân tán trong không gian $2D$. Mỗi điểm dữ liệu được xác định bởi hai tọa độ (hai mức độ biến thiên) hoàn toàn độc lập là Sepal Length và Sepal Width. Dữ liệu đã được dời tâm (mean-centered) về gốc tọa độ $(0, 0)$ nhưng vẫn giữ nguyên hình dáng phân bố gốc.

**Không gian đặc trưng (Quá trình xác định trục PCA):**

Ở đồ thị thứ hai, thuật toán đã tìm ra hệ quy chiếu mới gồm hai thành phần chính trực giao với nhau. Trục màu đỏ (PC1) đâm xuyên qua trục dài nhất của đám mây điểm. Trục màu xanh lá (PC2) vuông góc với PC1, đại diện cho chiều hẹp nhất của đám mây điểm. **Không gian nén (Trạng thái sau PCA - Giảm xuống 1D):**

Như thể hiện ở đồ thị thứ ba và đặc biệt là đồ thị kết hợp cuối cùng, toàn bộ đám mây điểm $2D$ màu xanh đã được chiếu trực giao (thể hiện qua các đường đứt nét màu xám) xuống một đường thẳng duy nhất là trục PC1 (màu đỏ).

**Kết quả:**

Dữ liệu từ không gian $2D$ đã bị "ép phẳng" xuống không gian $1D$ (các điểm màu tím/cam). Lúc này, tọa độ theo trục PC2 đã bị loại bỏ hoàn toàn. Các điểm dữ liệu mới chỉ còn một giá trị vô hướng duy nhất đại diện cho vị trí của chúng trên trục PC1. Mặc dù số chiều giảm đi một nửa,

khoảng cách tương đối và sự phân cụm giữa các điểm dọc theo trục PC1 vẫn được bảo toàn gần như nguyên vẹn so với không gian gốc.

## 3.2 Nhận xét về phương sai

**Tối đa hóa phương sai trên PC1:**

Theo nguyên lý toán học của PCA, thành phần chính thứ nhất (PC1) (đã được sắp xếp giảm dần theo eigenvalue) luôn là hướng trải dài nhất của dữ liệu. Biểu đồ kết hợp cho thấy rõ các điểm dữ liệu phân tán rất rộng dọc theo trục màu đỏ. Việc giữ lại PC1 đồng nghĩa với việc ta đã giữ lại được lượng phương sai (hay lượng thông tin) lớn nhất của tập dữ liệu ban đầu.

**Đánh đổi phương sai và sai số tái tạo:**

Thành phần chính thứ hai (PC2 - trục màu xanh lá) là hướng có độ phân tán dữ liệu hẹp nhất. Khoảng cách từ các điểm màu xanh (dữ liệu gốc) dóng vuông góc xuống trục màu đỏ (các đường đứt nét màu xám) chính là độ lớn của phương sai theo hướng PC2.

Khi nén xuống $1D$, ta chấp nhận vứt bỏ phương sai dọc theo trục PC2 này. Các đoạn thẳng đứt nét màu xám chính là sai số tái tạo (reconstruction error) hay lượng thông tin bị mất mát.

Kết luận: Vì đám mây điểm khá "dẹt" (phương sai theo hướng PC2 nhỏ), nên các đoạn đứt nét này khá ngắn. Điều này minh họa sự thành công của PCA: ta có thể giảm 50% khối lượng dữ liệu (từ $2D$ xuống $1D$) nhưng lượng thông tin/phương sai bị mất đi là khá nhỏ và gần như không làm thay đổi cấu trúc tổng thể của dữ liệu.

# 4 Câu 4 So sánh

```
=== Chiếu dữ liệu xuống không gian 2 chiều ===
So sánh PCA tự cài đặt với PCA của scikit-learn
1. Tốc độ chạy
Thời gian Numpy tự code : 0.000283 giây
Thời gian Scikit-learn  : 0.001747 giây

--------------------------------------------------

2. Trị riêng (Eigenvalues) của 2 phương pháp:
Numpy Tự Code (Top 2)  : [4.224841 0.242244]
Scikit-learn (Top 2)   : [4.224841 0.242244]
=> Sai số lớn nhất (top 2)       : 0.000000000000000

--------------------------------------------------

3. Dữ liệu sau giảm chiều (Projected Data) của 2 phương pháp:
Numpy Tự Code (5 mẫu đầu tiên):
[[ 2.684207  0.326607]
 [ 2.715391 -0.169557]
 [ 2.88982  -0.137346]
 [ 2.746437 -0.311124]
 [ 2.728593  0.333925]]

Scikit-learn (5 mẫu đầu tiên):
[[ 2.684207  0.326607]
 [ 2.715391 -0.169557]
 [ 2.88982  -0.137346]
 [ 2.746437 -0.311124]
 [ 2.728593  0.333925]]

=> Sai số lớn nhất trên toàn bộ 150 mẫu: 0.000000000000000

============================================================
```

Hình 8: So sánh tốc độ chạy và sai số giữa PCA khi tự code và dùng thư viện

## 4.1 Đánh giá về Tốc độ chạy

**Hiện tượng biến động về thời gian:** Việc thời gian chạy thay đổi ở mỗi lần run là do CPU phải xử lý đa nhiệm (background tasks của Windows, các tiến trình ngầm). Vì thuật toán chạy quá nhanh (chỉ mất chưa tới 1 mili-giây), các yếu tố như độ trễ hệ điều hành, trạng thái cache của CPU, hoặc trình dọn rác (Garbage Collector) của Python sẽ gây ra sự sai lệch nhỏ khi dùng hàm đo thời gian time.perf_counter().

**So sánh Numpy và scikit-learn:**

Dựa trên file kết quả, thời gian của NumPy (khoảng 0.0002s đến 0.0004s) luôn nhanh hơn Scikit-learn (khoảng 0.0009s đến 0.0027s). Nguyên nhân: NumPy nhanh hơn trong trường hợp này chỉ vì tập dữ liệu Iris quá nhỏ (150 mẫu, 4 đặc trưng). Thuật toán tự code chỉ thực hiện vài phép nhân ma trận cơ bản và giải phương trình trên ma trận kích thước nhỏ 4x4. Ngược lại,

scikit-learn là thư viện chuẩn công nghiệp. Trước khi tính toán thực sự, nó phải chạy qua các bước "overhead" (chi phí ngầm) như: kiểm tra tính hợp lệ của input (validation), rẽ nhánh để chọn thuật toán SVD phù hợp, và cấp phát bộ nhớ. Với dữ liệu nhỏ, thời gian chạy các bước kiểm tra này còn lâu hơn thời gian làm toán. Tuy nhiên, nếu áp dụng trên Big Data (ví dụ 1 triệu dòng), thuật toán SVD được tối ưu hóa ở tầng C/Cython của scikit-learn sẽ hoàn toàn áp đảo NumPy.

## 4.2 Đánh giá về Sai số giữa hai cách

**Tính chính xác tuyệt đối**: Xuyên suốt quá trình giảm chiều từ k=1 đến k=4, sai số lớn nhất giữa hai phương pháp (đối với cả Trị riêng và Ma trận dữ liệu chiếu) đều bằng chính xác 0.000000000000000 (chính xác đến 15 chữ số thập phân).

**Nhận xét đại số tuyến tính**: Kết quả sai số bằng 0 chứng minh đanh thép rằng phương pháp tiếp cận trực tiếp thông qua Ma trận hiệp phương sai (Covariance Matrix) tự cài đặt và phương pháp tối ưu Phân tích suy biến (SVD) của scikit-learn là tương đương nhau hoàn toàn về mặt toán học. Việc xử lý hiện tượng đa trị của vector riêng (Sign Ambiguity) trong mã nguồn đã triệt tiêu mọi khác biệt về chiều vector, bảo toàn tuyệt đối không gian biểu diễn dữ liệu của thuật toán PCA.

**Lưu ý về sai số dấu phẩy động (Floating-point error):** Mặc dù sai số tính toán trả về xấp xỉ 0.0 (chính xác đến 15 chữ số thập phân), nhưng về mặt bản chất hệ thống máy tính, một sai số vi mô (cỡ 1e-16) vẫn luôn tồn tại. Sự sai lệch cực nhỏ này phát sinh từ việc thuật toán tự cài đặt phải thực hiện phép nhân ma trận (tính $X^T X$) làm khuếch đại sai số biểu diễn số thực, trong khi thư viện scikit-learn sử dụng phép phân tích suy biến SVD trực tiếp để giảm thiểu chi phí hao hụt số học này. Tuy nhiên, sai số này là hoàn toàn không đáng kể và không làm ảnh hưởng đến tính đúng đắn của không gian trực giao.

# 5 Câu 5 Ứng dụng PCA vào bài toán nén và tái tạo ảnh

Hình 9: So sánh ảnh gốc và các ảnh được tái tạo

## 5.1 Ảnh hưởng của số lượng thành phần chính ($k$) đối với chất lượng ảnh:

**Khi $k$ rất nhỏ ($k = 2, 5$):** Ảnh tái tạo chỉ giữ lại được các đặc trưng tần số thấp nhất (low-frequency features). Cụ thể, ảnh ở $k = 2$ chỉ là những mảng khối sọc mờ ảo, MSE ở mức cao nhất (0.1816). Ở $k = 5$, hình khối của khuôn mặt bắt đầu xuất hiện nhưng vẫn bị mất mát toàn bộ chi tiết.

**Khi $k$ tăng dần ($k = 10, 20, 50$):** Chất lượng ảnh được cải thiện rõ rệt. Với $k = 50$, các chi tiết sắc nét (tần số cao) như nếp nhăn, viền mắt kính, hay kết cấu râu tóc đã được phục hồi tốt. Chỉ số MSE giảm mạnh, chứng tỏ thuật toán đang thu hồi lại được những phương sai mang tính quyết định của bức ảnh.

**Khi $k$ lớn ($k = 100, 200$):** Ảnh tái tạo đạt đến mức độ sắc nét gần như tương đương hoàn toàn với ảnh gốc khi quan sát bằng mắt thường. Lỗi MSE hội tụ về mức rất nhỏ ($0.156x$). Những thành phần chính còn lại bị cắt bỏ lúc này thực chất phần lớn là nhiễu (noise) hoặc các dao động pixel cực nhỏ không mang lại giá trị thị giác.

## 5.2 Sự đánh đổi (Trade-off) giữa Độ nén và Chất lượng ảnh:

**Bài toán minh họa rõ nét nguyên lý cốt lõi của việc nén dữ liệu có tổn hao:** Nén càng mạnh ($k$ càng nhỏ), dung lượng lưu trữ càng được tối ưu nhưng chất lượng ảnh càng suy giảm nghiêm trọng.

Tuy nhiên, PCA cho thấy hiệu suất nén tuyệt vời nhờ tận dụng tính dư thừa dữ liệu (data redundancy). Mặc dù bức ảnh có hàng trăm pixel theo chiều ngang, chỉ cần giữ lại $k = 50$ hoặc $k = 100$ thành phần chính (tương đương với việc chỉ dùng một tỷ lệ nhỏ dung lượng gốc), ta đã bảo toàn được hầu hết cấu trúc không gian và phương sai của hình ảnh.

# 6 Câu hỏi Bonus

## 6.1 So sánh PCA và t-SNE, Linear Discriminant Analysis

Trong các bài toán giảm chiều dữ liệu (Dimensionality Reduction), PCA là thuật toán nền tảng. Tuy nhiên, tùy thuộc vào đặc tính của tập dữ liệu và mục tiêu bài toán, các thuật toán khác như t-SNE hoặc LDA có thể mang lại hiệu quả vượt trội hơn. Dưới đây là bảng so sánh tổng quan và phân tích chi tiết:

Bảng 1: So sánh các kỹ thuật giảm chiều dữ liệu: PCA, t-SNE và LDA

| **Tiêu chí** | **PCA** | **t-SNE** | **LDA** |
|---|---|---|---|
| **Bản chất học** | Không giám sát (Unsupervised) | Không giám sát (Unsupervised) | Có giám sát (Supervised) |
| **Tính chất** | Tuyến tính (Linear) | Phi tuyến tính (Non-linear) | Tuyến tính (Linear) |
| **Mục tiêu tối ưu** | Tối đa hóa phương sai (giữ lại cấu trúc toàn cục). | Bảo toàn khoảng cách cục bộ (giữ các điểm giống nhau ở gần nhau). | Tối đa hóa khoảng cách giữa các lớp, tối thiểu hóa phương sai trong cùng lớp. |
| **Ưu điểm** | Nhanh, kết quả cố định (deterministic), tính toán được ma trận chiếu ngược. | Trực quan hóa dữ liệu phân cụm phi tuyến cực kỳ xuất sắc (2D/3D). | Rất mạnh mẽ trong bước tiền xử lý cho bài toán phân loại (Classification). |
| **Nhược điểm** | Kém hiệu quả với dữ liệu phi tuyến tính phức tạp (VD: cuộn Swiss Roll). | Chạy chậm, kết quả thay đổi sau mỗi lần chạy, không dùng để nén dữ liệu tổng quát. | Bắt buộc phải có nhãn (labels), số chiều giảm tối đa bị giới hạn bởi số lớp ($C-1$). |

### 6.1.1 Phân tích PCA và t-SNE

**Về mặt hình học:** PCA cố gắng tìm một mặt phẳng thẳng (trục tuyến tính) đâm xuyên qua dữ liệu sao cho các điểm trải dài nhất có thể. Tuy nhiên, nếu dữ liệu cuộn tròn hoặc có hình đa tạp phức tạp, PCA sẽ làm các điểm bị chồng chéo lên nhau. Ngược lại, t-SNE sử dụng xác suất để "mở cuộn" dữ liệu phi tuyến, ưu tiên duy trì cấu trúc cục bộ (Local Structure).

**Về mặt ứng dụng:** Nếu mục tiêu là nén dữ liệu để lưu trữ hoặc tăng tốc độ cho model (như bài toán nén ảnh ở Câu 5), bắt buộc phải dùng PCA vì nó có ma trận chuyển vị để khôi phục ảnh. t-SNE không tạo ra hàm ánh xạ rõ ràng nên không thể áp dụng cho dữ liệu mới (out-of-sample) và không thể dùng để nén/khôi phục ảnh. Tác dụng lớn nhất của t-SNE là trực quan hóa dữ liệu trên mặt phẳng $2D$.

### 6.1.2 Phân tích PCA và LDA

**Vấn đề về nhãn (Label):** Nhược điểm lớn nhất của PCA trong bài toán phân loại là nó hoàn toàn "mù" trước nhãn dữ liệu. PCA chỉ quan tâm hướng nào dữ liệu phân tán rộng nhất. Đôi khi, hướng phương sai lớn nhất lại gộp chung 2 lớp khác nhau vào làm một, gây khó khăn cho việc phân loại. **Ưu điểm của LDA:** Bù lại, LDA là thuật toán có giám sát. Nó sử dụng thông tin nhãn lớp để tính toán ma trận phân tán giữa các lớp (Between-class scatter) và trong nội bộ lớp (Within-class scatter). LDA sẽ cố tình xoay trục tọa độ sao cho các mẫu thuộc cùng một lớp co cụm lại với nhau, trong khi các lớp khác nhau bị đẩy ra xa nhất có thể. Do đó, trong các pipeline máy học học có giám sát, LDA thường cho độ chính xác cao hơn PCA. Với LDA, nó tận dụng nhãn lớp để giải quyết đồng thời hai mục tiêu toán học gồm: Tối thiểu hóa độ phân tán trong nội bộ lớp (Minimize Within-class Scatter - $S_W$) Ép các điểm dữ liệu thuộc cùng một lớp (ví dụ: cùng một người) chiếu xuống sát lại với nhau thành một cụm chặt chẽ (phương sai cục bộ đạt cực tiểu) và Tối đa hóa khoảng cách giữa các lớp (Maximize Between-class Scatter - $S_B$) Đẩy tâm (mean) của các cụm dữ liệu khác nhau ra xa nhau nhất có thể.

## 6.2 Áp dụng PCA cho bài toán Giảm nhiễu tín hiệu (Signal/Image Denoising)

Bên cạnh khả năng nén dữ liệu, PCA còn là một công cụ mạnh mẽ để lọc nhiễu. Trong không gian đặc trưng của hình ảnh, các thành phần chính (Principal Components) đầu tiên mang trị riêng cực đại sẽ nắm giữ cấu trúc cốt lõi, hình khối và ánh sáng của vật thể. Ngược lại, các thành phần chính ở cuối mang trị riêng rất nhỏ thường chỉ đại diện cho nhiễu hạt (noise) hoặc các biến thiên ngẫu nhiên tần số cao không mang giá trị thông tin. Bằng cách thực hiện phép chiếu PCA và chỉ giữ lại $k$ thành phần chính đầu tiên, ta đã loại bỏ đi các trục không gian chứa nhiễu. Khi tái tạo lại ma trận ảnh từ không gian giảm chiều này:

$$X_{denoised} = (X_{reduced} \cdot U_k^T) + \mu$$

Dữ liệu thu được không những có kích thước tối ưu mà còn được làm mịn, khử sạch các nhiễu ngẫu nhiên so với ma trận gốc $X$. Đây là nền tảng cho nhiều bộ lọc nhiễu truyền thống trong lĩnh vực Xử lý ảnh kỹ thuật số (Digital Image Processing).

**Ví dụ từ kết quả câu 5:** Quan sát bảng số liệu Lỗi tái tạo (MSE) khi thực hiện trên ảnh chân dung, ta thấy một minh chứng rõ nét cho cơ chế lọc nhiễu này:Từ $k = 2$ đến $k = 50$: Lỗi MSE giảm rất mạnh. Các vector riêng ở nhóm này đang làm nhiệm vụ khôi phục lại các đặc điểm sinh trắc học (mắt, mũi, miệng,...) và kết cấu quan trọng. Từ $k = 100$ đến $k = 200$: Lỗi MSE gần như bão hòa, chỉ giảm một lượng cực kỳ vi mô từ 0.1563 xuống 0.1562 (chênh lệch 0.0001). Điều này khẳng định rằng hàng trăm vector riêng (eigenvectors) từ vị trí 100 trở đi hoàn toàn không đóng góp vào việc định hình cấu trúc khuôn mặt, mà chúng đang biểu diễn những điểm nhiễu li ti, các pixel bị sai lệch do cảm biến máy ảnh. Việc ta chỉ khôi phục ảnh ở mức $k = 50$ hoặc $k = 100$ ($X_{reconstructed} = X_{reduced} \cdot U_k^T + \mu$) thực chất đã thực hiện một bộ lọc thông thấp (low-pass filter). Bức ảnh tái tạo tuy mất đi một vài chiều dữ liệu nhưng lại thu được bề mặt mịn màng và "sạch nhiễu" hơn rất nhiều so với ma trận gốc bị nhiễu.

## 6.3 Áp dụng PCA cho bài toán nhận diện khuôn mặt

Nhận dạng khuôn mặt là một trong những ứng dụng kinh điển nhất của thuật toán PCA trong lĩnh vực Thị giác máy tính (Computer Vision), thường được biết đến với tên gọi Eigenfaces. Thay vì so sánh trực tiếp từng pixel (dễ bị nhiễu do ánh sáng, biểu cảm), hệ thống sử dụng PCA để chuyển đổi hình ảnh sang một không gian có chiều thấp hơn, nơi các đặc điểm sinh trắc học cốt lõi được bảo toàn.

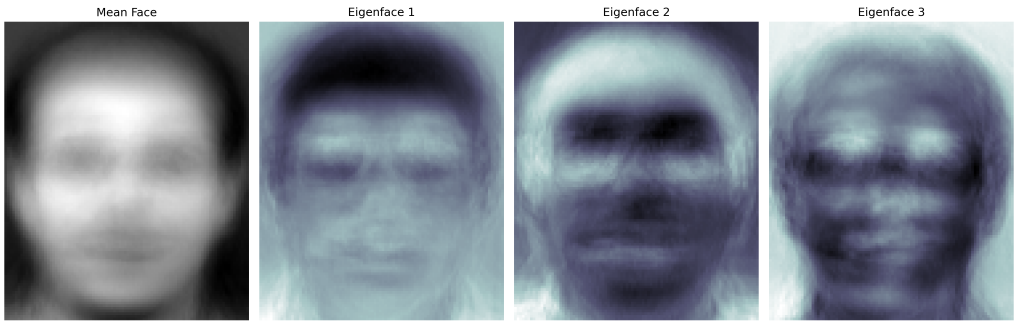
Trong thử nghiệm này, hệ thống sử dụng tập dữ liệu chuẩn mực ATT Database of Faces. Dữ liệu bao gồm 400 bức ảnh của 40 cá nhân. Mỗi bức ảnh grayscale có kích thước $112 \times 92$ pixel.

- Tiền xử lý: Mỗi bức ảnh 2D được trải phẳng (flatten) thành một vector 1D có kích thước 10304 chiều.
- Phân chia dữ liệu: Hệ thống trích xuất 8 ảnh đầu tiên của mỗi người để làm tập Huấn luyện (Train) và 2 ảnh cuối để làm tập Kiểm thử (Test). Từ đó ta có: Ma trận huấn luyện $X_{train}$ có kích thước: $320 \times 10304$
  Ma trận kiểm thử $X_{test}$ có kích thước: $80 \times 10304$

## 6.4 Giải quyết vấn đề về bài toán tràn bộ nhớ bằng thủ thuật Turk Pentland

Với ma trận $X_{train}$ kích thước $320 \times 10304$, việc tính toán trực tiếp ma trận hiệp phương sai $C = X^T \cdot X$ sẽ tạo ra một ma trận khổng lồ kích thước $10304 \times 10304$. Việc giải phương trình tìm trị riêng cho ma trận này là bất khả thi đối với cấu hình máy tính thông thường do tràn RAM. Để giải quyết, đồ án áp dụng thủ thuật toán học của Turk Pentland (1991):

- Thay vì tính $C$, hệ thống tính ma trận xấp xỉ $L = X \cdot X^T$ với kích thước siêu nhỏ chỉ $320 \times 320$.
- Sau khi tìm được vector riêng $v$ của $L$, hệ thống khôi phục các vector riêng thực sự của không gian ảnh (các Eigenfaces) bằng phép chiếu ngược: $U = X^T \cdot v$.
- Cuối cùng, hệ thống lọc ra $k = 50$ thành phần chính (Eigenfaces) mang trị riêng cực đại. Ma trận $X_{train}$ lúc này được nén thành công từ 10304 chiều xuống chỉ còn 50 chiều (kích thước nén: $320 \times 50$).




Hình 10: Phân rã khuôn mặt thành các đặc trưng toán học PCA.

### 6.5 Tích hợp lọc nhiễu tín hiệu

Bên cạnh việc giảm chiều để nhận dạng, quá trình giữ lại $k = 50$ Eigenfaces hàng đầu và loại bỏ hơn 10000 chiều phía sau bản chất chính là một bộ lọc nhiễu thông thấp (Low-pass filter). Các vector riêng mang trị riêng lớn ở đầu đại diện cho cấu trúc xương, hốc mắt, ánh sáng tổng thể. Ngược lại, hàng ngàn vector riêng ở phần đuôi bị vứt bỏ thực chất chỉ chứa các dao động tần số cao, nhiễu hạt (noise) của cảm biến máy ảnh. Việc chiếu dữ liệu qua không gian Eigenfaces giúp các đặc trưng trích xuất được làm mịn và miễn nhiễm với các nhiễu loạn nhỏ.

### 6.6 Đánh giá kết quả Kiểm thử và Tinh chỉnh tham số k-NN

Trong giai đoạn kiểm thử, 80 bức ảnh Test hoàn toàn mới được trừ đi khuôn mặt trung bình (Mean Face) và chiếu xuống không gian 50 chiều. Hệ thống sử dụng thuật toán K-Nearest Neighbors (K-NN) dựa trên khoảng cách Euclidean để phân loại. Dựa trên kết quả thực thi (được đính kèm trong log terminal), ta có bảng thống kê sự biến thiên của độ chính xác theo tham số $K_{NN}$:

- Khi $K_{NN} = 1$ và $K_{NN} = 2$: Độ chính xác đạt cực đại 95
- Khi $K_{NN} \geq 3$: Độ chính xác sụt giảm nhẹ xuống 93.75% (dự đoán đúng 75/80 ảnh) và duy trì đi ngang. K càng tăng, độ chính xác càng giảm dần

Sự sụt giảm độ chính xác khi tăng tham số $K_{NN}$ phản ánh đúng tính chất phân bố không gian của tập dữ liệu ATT. Do mỗi lớp (mỗi cá nhân) chỉ có 8 mẫu huấn luyện, mật độ dữ liệu trong không gian PCA là tương đối thưa thớt.

- Khi dùng $1 - NN$, thuật toán chỉ tin tưởng vào điểm láng giềng có khoảng cách tuyệt đối gần nhất, giúp bắt trúng các đặc trưng sắc thái sát nhất với ảnh gốc.
- Khi mở rộng vùng tìm kiếm ($K_{NN} \geq 3$), do số lượng mẫu cùng lớp bị giới hạn, hệ thống buộc phải lôi kéo thêm các mẫu thuộc các cá nhân khác (nhưng có cấu trúc mặt hao hao giống) vào để biểu quyết. Quá trình biểu quyết đa số này vô tình mang theo "nhiễu biểu quyết" (voting noise), khiến mẫu kiểm thử bị phân loại nhầm. Để cải tiến thì ta có thể sử dụng khoảng cách có trọng số để giảm ảnh hưởng với các mẫu ở xa

```
Đang đọc dữ liệu trong  ./eigenfaces_dataset
Kích thước 1 bức ảnh gốc: (112, 92)
Tập Train (X_train) : 320 ảnh, ma trận kích thước (320, 10304)
Tập Test  (X_test)  : 80 ảnh, ma trận kích thước (80, 10304)
Trích xuất thành công 50 mặt đặc trưng
Kích thước X_train sau khi nén: (320, 50)

 Bước 3: Kiểm thử

 ------------------------------------------------

KNN = 1
Tổng số ảnh Test : 80
Dự đoán đúng     : 76
Độ chính xác     : 95.0000%

================================================
KNN = 2
Tổng số ảnh Test : 80
Dự đoán đúng     : 76
Độ chính xác     : 95.0000%

================================================
KNN = 3
Tổng số ảnh Test : 80
Dự đoán đúng     : 75
Độ chính xác     : 93.7500%

================================================
KNN = 4
Tổng số ảnh Test : 80
Dự đoán đúng     : 75
Độ chính xác     : 93.7500%

================================================
KNN = 5
Tổng số ảnh Test : 80
Dự đoán đúng     : 75
Độ chính xác     : 93.7500%

------------------------------------------------
```

Hình 11: Kết quả kiểm thử sau quá trình huấn luyện

**Kết luận:** Đối với bài toán nhận dạng khuôn mặt trên tập dữ liệu có mật độ mẫu phân bố mỏng, sự kết hợp giữa không gian nén Eigenfaces ($k_{PCA} = 50$) và chiến lược láng giềng gần nhất tuyệt đối ($1 - NN$) mang lại hiệu năng phân loại xuất sắc nhất (95%).

# Tài liệu

[1] Matthew Turk and Alex Pentland. *Eigenfaces for Recognition.* Journal of Cognitive Neuroscience, 3(1), 71-86, MIT Press, 1991. (Nguồn tham khảo lý thuyết cốt lõi cho thuật toán nhận dạng khuôn mặt và thủ thuật giảm chiều ma trận).

[2] Christopher M. Bishop. *Pattern Recognition and Machine Learning.* Springer, 2006. (Tham khảo nền tảng toán học về Principal Component Analysis, Lỗi tái tạo MSE và Linear Discriminant Analysis).

[3] AT&T Laboratories Cambridge. *The ORL Database of Faces (AT&T Face Database).* [Online]. Available trên Kaggle: `https://www.kaggle.com/datasets/kasikrit/att-database-of-faces` (Sử dụng cho thử nghiệm nén ảnh và nhận dạng khuôn mặt Eigenfaces).

[4] Gary B. Huang, Manu Ramesh, Tamara Berg, and Erik Learned-Miller. *Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments.* Technical Report 07-49, University of Massachusetts, Amherst, 2007. [Online]. Available: `http://vis-www.cs.umass.edu/lfw/` (Sử dụng cho thực nghiệm đánh giá nhiễu tín hiệu và lỗi tái tạo không gian tuyến tính).

[5] Ronald A. Fisher. *The Use of Multiple Measurements in Taxonomic Problems.* Annals of Eugenics, 7(2), 179-188, 1936. [Online]. Available trên UCI Machine Learning Repository (Iris Dataset): `https://archive.ics.uci.edu/ml/datasets/iris` (Sử dụng cho các thử nghiệm phân loại và giảm chiều dữ liệu cơ bản).

[6] Peter N. Belhumeur, João P. Hespanha, and David J. Kriegman. *Eigenfaces vs. Fisherfaces: Recognition Using Class Specific Linear Projection.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 19(7), 711-720, 1997. [Online]. Available trên Kaggle (Yale Face Database): `https://www.kaggle.com/datasets/zerghamwarraich1/yale-images-dataset-for-image-classifications`

[7] Karl Pearson. *On Lines and Planes of Closest Fit to Systems of Points in Space.* The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science, 2(11), 559-572, 1901. (Bài báo khoa học gốc đặt nền móng toán học đầu tiên cho thuật toán Phân tích thành phần chính - PCA).

[8] Jonathon Shlens. *A Tutorial on Principal Component Analysis.* arXiv preprint arXiv:1404.1100, 2014. [Online]. Available: `https://arxiv.org/abs/1404.1100` (Tài liệu hướng dẫn chi tiết và chuẩn mực nhất về cơ sở Đại số tuyến tính, ma trận hiệp phương sai và ứng dụng của PCA trong khoa học máy tính).

[9] Gilbert Strang. *Introduction to Linear Algebra.* 5th ed. Wellesley, MA: Wellesley-Cambridge Press, 2016. (Sách giáo khoa kinh điển về Đại số tuyến tính, tài liệu tham khảo chính cho các phép chiếu trực giao, trị riêng (eigenvalues) và vector riêng (eigenvectors)).